🤖 AI Code Review — Murajea

File: UserService.cs
Date: 2026-03-26 02:53:10

📊 Summary

Total Issues	6
Critical Issues	2
Warnings	4
Functions Reviewed	3

🔍 Detailed Review

📌 CreateUser

Line 9

Code:

```
public User CreateUser(string name, string email)
        {
            // مشكلة: مفيش validation للـ email
            // مشكلة: مفيش null check للـ name
            var user = new User
            {
                Name = name,
                Email = email,
                CreatedAt = DateTime.Now
            };

            _users.Add(user);
            return user;
        }
```

Issues:

🔴 Critical: The function does not validate the email address before creating a new user

⚠️ Warning: The function does not check for null or empty name before creating a new user

💡 **Suggestion:**
Implement input validation for both name and email, and consider using a regex to validate the email format and check for null or empty strings for the name

📌 GetUserById

Line 24

Code:

```
public User GetUserById(int id)
        {
            // مشكلة: ممكن ترجع null من غير ما تتحقق
            return _users.FirstOrDefault(u => u.Id == id);
```

```
        }
```

Issues:

⚠️ Warning: The function may return null without being checked, potentially causing a NullReferenceException.

💡 **Suggestion:**
Consider adding a null check or using the nullable reference type feature to handle the possibility of a null return value, and provide a meaningful default value or throw a custom exception if the user is not found.

📌 UpdateUserEmailAsync

Line 30

Code:

```
public async Task<bool> UpdateUserEmailAsync(int id, string newEmail)
        {
            // مشكلة: async method غير من CancellationToken
            var user = GetUserById(id);
            if (user == null) return false;

            user.Email = newEmail;
            await Task.Delay(100);
            return true;
        }
```

Issues:

🔴 Critical: The method UpdateUserEmailAsync is missing a CancellationToken parameter which can lead to unhandled OperationCanceledExceptions and make the method non-cancelable.

⚠️ Warning: The method UpdateUserEmailAsync is using a synchronous call to GetUserById which can block the calling thread and potentially lead to deadlocks.

⚠️ Warning: The method UpdateUserEmailAsync is using a Task.Delay which can be unnecessary and may indicate a missing or incomplete implementation of an asynchronous operation.

💡 **Suggestion:**
Consider adding a CancellationToken parameter to the UpdateUserEmailAsync method, using an asynchronous version of GetUserById, and replacing Task.Delay with the actual asynchronous operation that is being delayed.